

Difference Between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

JPA stands for Java Persistence API.

It is a specification that defines standards for storing and managing Java objects in relational databases.

Characteristics:

- Provides annotations and interfaces
- Does not provide implementation
- Requires an ORM provider

Examples:

- Hibernate
- EclipseLink

Hibernate

Hibernate is an ORM framework that implements JPA specifications.

Responsibilities:

- Object-relational mapping
- SQL generation
- Database communication
- Transaction management

Hibernate converts:

Java Object ----> Database Table

Spring Data JPA

Spring Data JPA is an abstraction layer built on top of JPA.

It reduces boilerplate code by providing repository interfaces.

Advantages:

- Automatic CRUD operations
- Simplified database access
- Automatic transaction management

Code Comparison

Hibernate Implementation

Hibernate requires manual session and transaction handling.

Example:

```
Session session =
```

```
factory.openSession();
```

```
Transaction tx =
```

```
session.beginTransaction();
```

```
session.save(employee);
```

```
tx.commit();
```

```
session.close()
```

Developer manages:

- Session

- Transaction
- Commit
- Rollback

Spring Data JPA Implementation

Spring Data JPA uses repositories.

Repository:

```
public interface EmployeeRepository
    extends JpaRepository<Employee, Integer>{
}
```

Saving employee:

```
employeeRepository.save(employee);
```

Spring automatically manages:

- Session
- Transaction
- SQL generation

Comparison Table

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Abstraction Layer
Implementation	No	Yes	Uses Hibernate
SQL Generation	No	Yes	Through Hibernate
Transaction Management	No	Manual	Automatic
Code Required	Medium	More	Less
Mapping	Annotations	XML/Annotations	JPA Annotations